Auto-Root Login

Automatically switch to the root user, with a password prompt, every time you SSH into an Ubuntu server.

Item	Detail
Environment	Ubuntu on AWS EC2
SSH client	MobaXterm
Method	One line in the user's .bashrc that runs **su -** on login
Setup options	Automated script (setup.sh) or manual commands
Intended use	Labs and learning. Not for production servers.

1. Overview

By default, getting root on an Ubuntu server means typing `sudo su` after every login. This project removes that step. When you connect as `ubuntu`, the server runs `su -` automatically, asks for **root's password**, and opens a root login shell.

The result is a login that goes straight from SSH to a root prompt, protected by a password, with no extra commands to type.

> **Note:** Running as root permanently is risky. One wrong command, such as a misplaced **rm -rf**, can damage the whole system. This setup is meant for labs and learning, not for production machines.

2. Project Contents

File	Purpose
README.md	Project overview and instructions for GitHub.
setup.sh	Automated installer with install, status, uninstall and dry-run modes.
docs/Auto-Root-Login-Guide.pdf	This document.
images/terminal-output.png	Screenshot of the setup commands running on the server.

3. How It Works

When you log in, Bash reads `~/.bashrc`. We add one line at the end of that file:

```
[[ $- == *i* && $EUID -ne 0 ]] && exec su -
```

Part	What it does
`$- == *i*`	True only in **interactive** terminals. This keeps MobaXterm's SFTP sidebar and other non-interactive sessions from hanging.
`$EUID -ne 0`	True only if you are **not already root**. This prevents the line from looping once you are root.
`&&`	Runs the next command only if both checks above passed.
`exec su -`	Replaces the current shell with a root **login** shell. `su -` asks for root's password. Because of `exec`, typing `exit` ends the session cleanly.

Login flow

Step	What happens
1	You connect through MobaXterm as ubuntu (authenticated by your key pair).
2	Bash starts an interactive shell and reads .bashrc.
3	The added line checks that the shell is interactive and that you are not root.
4	exec su - runs and shows a Password: prompt.
5	You enter root's password and land at a root prompt (root@host:~#).

Why su - and not sudo su?

On AWS EC2, the `ubuntu` user has **passwordless sudo** by default, so `sudo su` never asks for a password. `su -` asks for **root's own password**, which is what provides the password prompt. That is why this setup first gives root a password.

4. Requirements

Requirement	Details
Server	Ubuntu (tested on AWS EC2)
User	A normal user with sudo access (the default ubuntu user on EC2)
SSH client	MobaXterm, or any SSH client
Second tab	A second open terminal tab to test safely

5. Method A: Automated Setup (setup.sh)

The script does the same steps as the manual method, with backups, checks and a clear step-by-step display. Upload or copy setup.sh to the server, then run it as your normal user:

```
bash setup.sh
```

Command	What it does
`bash setup.sh`	Installs the auto-root login.
`bash setup.sh status`	Shows whether it is currently active.
`bash setup.sh uninstall`	Removes the line it added.
`bash setup.sh --dry-run`	Shows what would happen without changing anything.

What the installer does

Step	Action
1/5 Pre-flight checks	Confirms su, passwd and sudo access are available.
2/5 Set the root password	Asks whether to set or change root's password, then runs sudo passwd root.
3/5 Back up .bashrc	Saves a timestamped copy, for example .bashrc.bak.1791367523.
4/5 Add the line	Appends the auto-switch line (marked with a comment).
5/5 Verify	Confirms the line is present and prints it.

Safe to re-run. If the line is already installed, the script detects it and does not add it twice.

6. Method B: Manual Setup

Log in as `ubuntu` through MobaXterm. Your prompt looks like this:

```
ubuntu@ip-172-31-38-241:~$
```

Step 1: Set a password for root

On EC2, root has no password by default. Set one:

```
sudo passwd root
```

Enter the password twice. Nothing is shown while you type, which is normal. Expected output:

```
passwd: password updated successfully
```

Step 2: Add the auto-switch line

```
echo '[[ $- == *i* && $EUID -ne 0 ]] && exec su -' >> /home/ubuntu/.bashrc
```

Use the full path. If you are already root when you run this, ~ points to `/root` and the line would be added to the wrong file.

Step 3: Verify

```
tail -n 2 /home/ubuntu/.bashrc
```

The last line should be:

```
[[ $- == *i* && $EUID -ne 0 ]] && exec su -
```

Terminal output

The commands from Steps 1 to 3 as run on the server. The password was set, the line was added, and `tail` confirms it is the last line of .bashrc.

Figure 1: Setting the root password, adding the auto-switch line, and verifying it.

7. Testing

Keep your current tab open and test in a **new tab**. If something is misconfigured, the open tab is your way back in to fix it.

#	What to do	Expected result
1	Open a new SSH session as ubuntu	Connects normally
2	Watch the terminal	A Password: prompt appears automatically
3	Enter the root password	Prompt changes to root@ip-172-31-38-241:~#
4	Run: whoami	Prints: root
5	Check the MobaXterm file sidebar	Still loads normally

8. Undo

Using the script:

```
bash setup.sh uninstall
```

Or manually, which removes the last line of the file:

```
sed -i '$d' /home/ubuntu/.bashrc
```

Only use the manual command if the auto-switch line is still the **last line** of the file. Otherwise open `nano /home/ubuntu/.bashrc` and delete it by hand.

9. Troubleshooting

Problem	Cause	Fix
Kicked out right after entering the password	Wrong root password. exec closes the session on failure.	Use your open tab and reset it with: sudo passwd root
Locked out of every new session	The line is active but the root password is unknown or not set.	Fix from an already-open session. Otherwise see the recovery tip below.
MobaXterm file sidebar hangs	The interactive check is missing from the line.	Make sure the line contains $- == *i*
No password prompt appears	Line was not saved, or the wrong .bashrc was edited.	Run tail -n 2 /home/ubuntu/.bashrc, then reconnect with a fresh login.
Line ended up in root's file	Command run as root using ~ instead of the full path.	Use /home/ubuntu/.bashrc explicitly.

Recovery tip

If you are locked out and have no open session, connect with a standard SSH client and ask for a shell that skips startup files. The auto-switch line only runs in interactive login shells, so this should get you in:

```
ssh -i your-key.pem -t ubuntu@YOUR_SERVER_IP "bash --norc"
```

Then remove the line with `bash setup.sh uninstall` or the manual command from section 8.

10. Alternative Methods

Method	Pros	Cons
MobaXterm Execute command (Edit session, Advanced SSH settings, Execute command: su -)	No server changes. Easiest to undo.	Only applies to that saved MobaXterm session.
Direct root SSH login (PermitRootLogin yes in /etc/ssh/sshd_config)	Root is truly the default user.	Riskiest option. Root over SSH is a common attack target. On EC2 you must also check /etc/ssh/sshd_config.d/*.conf.
Use .bash_profile instead of .bashrc	Runs only on login shells.	Can change which startup files load. Test carefully.

11. Security Notes

• Use a **strong, unique root password**. Root accounts on public servers are heavily targeted.

• This setup does **not** change SSH authentication. You still connect with your key pair, and the password prompt comes from `su -` after you log in.

• For real systems, log in as a normal user and use `sudo` only when you need elevated rights.

• Never commit passwords, .pem key files, or private server details to a public repository.

12. FAQ

Question	Answer
Does this change who can log in over SSH?	No. SSH access is unchanged. This only affects what happens after a successful login.
Does it affect other users?	No. It edits one user's .bashrc only.
Can I use a different username?	Yes. Replace `ubuntu` with your username in the commands. The script uses the user who runs it.
Will scp and SFTP still work?	Yes. The interactive check means non-interactive sessions are skipped.